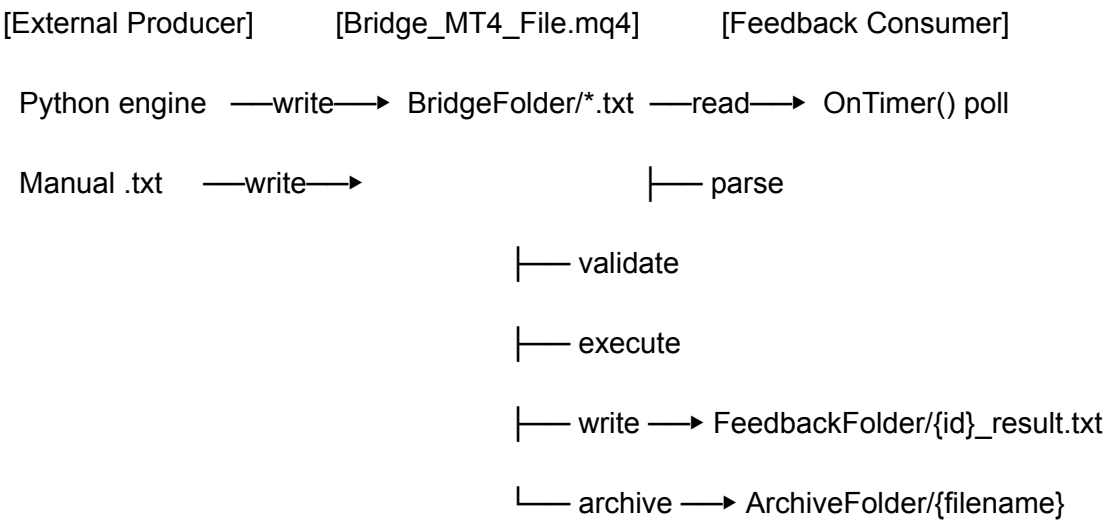
Design — Bridge_MT4_File.mq4

Architecture Summary

The EA is a single `.mq4` file with no external dependencies. It uses MT4's built-in file I/O, timer, and order functions. The design is deliberately simple: a poll loop reads files, validates, executes, writes feedback, and archives. The Python engine (future) writes the same files — the EA never knows or cares who created them.



Input Parameters

```
extern string BridgeFolder        = "C:\\bridge\\outgoing\\";

extern string FeedbackFolder      = "C:\\bridge\\incoming\\";

extern string ArchiveFolder       = "C:\\bridge\\archive\\";

extern bool   OnlyCurrentSymbol   = false;

extern bool   AskForConfirmation  = false;

extern double MaxLotsPerTrade     = 1.0;

extern double MaxSpread           = 3.0;      // in points
```

```
extern int    MagicNumberBase     = 202600;

extern int    PollIntervalSeconds = 1;

extern int    Slippage            = 3;       // in points

extern bool   VerboseLogging      = true;
```

All paths must end with `\\`. The EA logs them on init so you can verify they're correct before any files are processed.

File Protocol

Action File

- **Location**: `BridgeFolder`
- **Naming convention**: `{asset}_{YYYYMMDD}_{HHMMSS}_{seq}.txt`
  Example: `xauusd_20260523_120001_001.txt`
- **Format**: `key=value`, one pair per line, UTF-8 plain text

Key	Required	Description
`id`	Yes	Unique action identifier (matches filename without extension)
`asset`	Yes	Symbol, e.g. `XAUUSD`
`action`	Yes	`OPEN` or `CLOSE_ALL`
`side`	OPEN only	`BUY` or `SELL`
`size`	OPEN only	Lot size, e.g. `0.10`
`order_type`	OPEN only	`MARKET` (only supported type in v1)
`sl`	No	Stop loss price; blank = 0.0
`tp`	No	Take profit price; blank = 0.0

Key	Required	Description
`magic_number`	No	Override EA magic; blank = use `MagicNumberBase`
`valid_until`	No	ISO 8601 datetime; blank = always valid
`comment`	No	Order comment string

Feedback File

- **Location**: `FeedbackFolder`
- **Naming**: `{id}_result.txt`
- **Format**: same `key=value` structure

Key	Description
`id`	Mirrors action `id`
`status`	`FILLED` or `REJECTED`
`asset`	Symbol
`action`	Mirrors action type
`side`	Mirrors side
`size`	Mirrors size
`tickets`	Comma-separated ticket numbers (blank if rejected)
`avg_price`	Fill price (blank if rejected)
`message`	Human-readable outcome description
`error_code`	`0` = success, `1` = local risk rejection, `2` = MT4 error

---

Module Breakdown

`OnInit()`

1. Log configured paths.
2. Call `EventSetTimer(PollIntervalSeconds)`.
3. Return `INIT_SUCCEEDED`.

`OnDeinit(const int reason)`

1. Call `EventKillTimer()`.
2. Log shutdown.

`OnTimer()`

1. Build search mask: `BridgeFolder + "*.txt"`.
2. Use `FileFindFirst` / `FileFindNext` to iterate matching files.
3. For each file, call `ProcessActionFile(fullpath)`.
4. Call `FileFindClose(handle)`.

`ProcessActionFile(string path)`

1. Call `ReadActionFile(path, ...)` — returns `false` on I/O failure → log and skip (do not archive; retry next cycle).
2. Validate required fields (`id`, `action`, `asset`).
3. Check `OnlyCurrentSymbol` filter.
4. Branch on `action`:
    - `OPEN` → `ExecuteOpen(...)`
    - `CLOSE_ALL` → `ExecuteCloseAll(...)`
    - Unknown → reject with `message=UnknownAction`.
5. Call `ArchiveActionFile(path)`.

`ReadActionFile(string path, string &id, ...)`

- Opens file with `FILE_READ | FILE_TXT`.
- Reads line-by-line via `FileReadString`.
- Splits each line on `=` using `StringSplit`.
- Trims whitespace from keys and values (custom `StringTrim` helper).
- Populates out-parameters.
- Returns `true` on success, `false` if file cannot be opened.

> **MQL4 Note**: `StringSplit` splits on a single character. Values containing `=` (unlikely but possible) will be truncated. For v1 this is acceptable.

`ValidateOpen(string asset, double size, string side) → bool`

- Check `size <= MaxLotsPerTrade`.
- Check spread: `(Ask - Bid) / Point <= MaxSpread`.
- Check `side` is `BUY` or `SELL`.
- Check `valid_until` if non-empty (parse date components, compare with `TimeCurrent()`).
- On failure: call `WriteFeedback(...)` with `REJECTED` and return `false`.

`ExecuteOpen(string id, string asset, string side, double size, double sl, double tp, int magic, string comment)`

1. If `AskForConfirmation`: show `MessageBox` → on Cancel, write `REJECTED` and return.
2. Call `RefreshRates()`.
3. Determine price: `Ask` for BUY, `Bid` for SELL.
4. Determine op: `OP_BUY` or `OP_SELL`.
5. Call `OrderSend(asset, op, size, price, Slippage, sl, tp, comment, magic, 0, colour)`.
6. If `ticket > 0`: call `WriteFeedback` with `FILLED`, ticket, `OrderOpenPrice()`.
7. Else: call `WriteFeedback` with `REJECTED`, `error_code=2`, `GetLastError()` message.

`ExecuteCloseAll(string id, string asset, int magic)`

1. Loop `OrdersTotal()-1` downto `0`.
2. `OrderSelect(i, SELECT_BY_POS, MODE_TRADES)`.
3. Filter: `OrderSymbol() == resolvedSymbol` AND (magic == 0 OR `OrderMagicNumber() == magic`).
4. Close: `OrderClose(ticket, lots, closePrice, Slippage, colour)`.
5. Track `closedTickets[]`, `failCount`.
6. Write `FILLED` with joined ticket list, or `REJECTED` if nothing matched.

`WriteFeedback(...)`

- Opens `FeedbackFolder + id + "_result.txt"` with `FILE_WRITE | FILE_TXT`.
- Writes all fields line by line.
- Closes handle.

`ArchiveActionFile(string path)`

- Extracts filename from path.
- Calls `FileMove(path, ArchiveFolder + filename)`.
- On failure, calls `FileDelete(path)`.

`StringTrim(string s) → string`

- Manual helper: strip leading/trailing spaces and `\r` characters (MQL4 has no built-in trim).

Error Code Reference

Code	**Meaning**
0	Success
1	Local validation rejection (spread, lot size, symbol, expiry, user cancel)
2	MT4 `OrderSend` / `OrderClose` error — see `message` for `GetLastError()` value

Folder Layout (MT4 Host Machine)

C:\bridge\

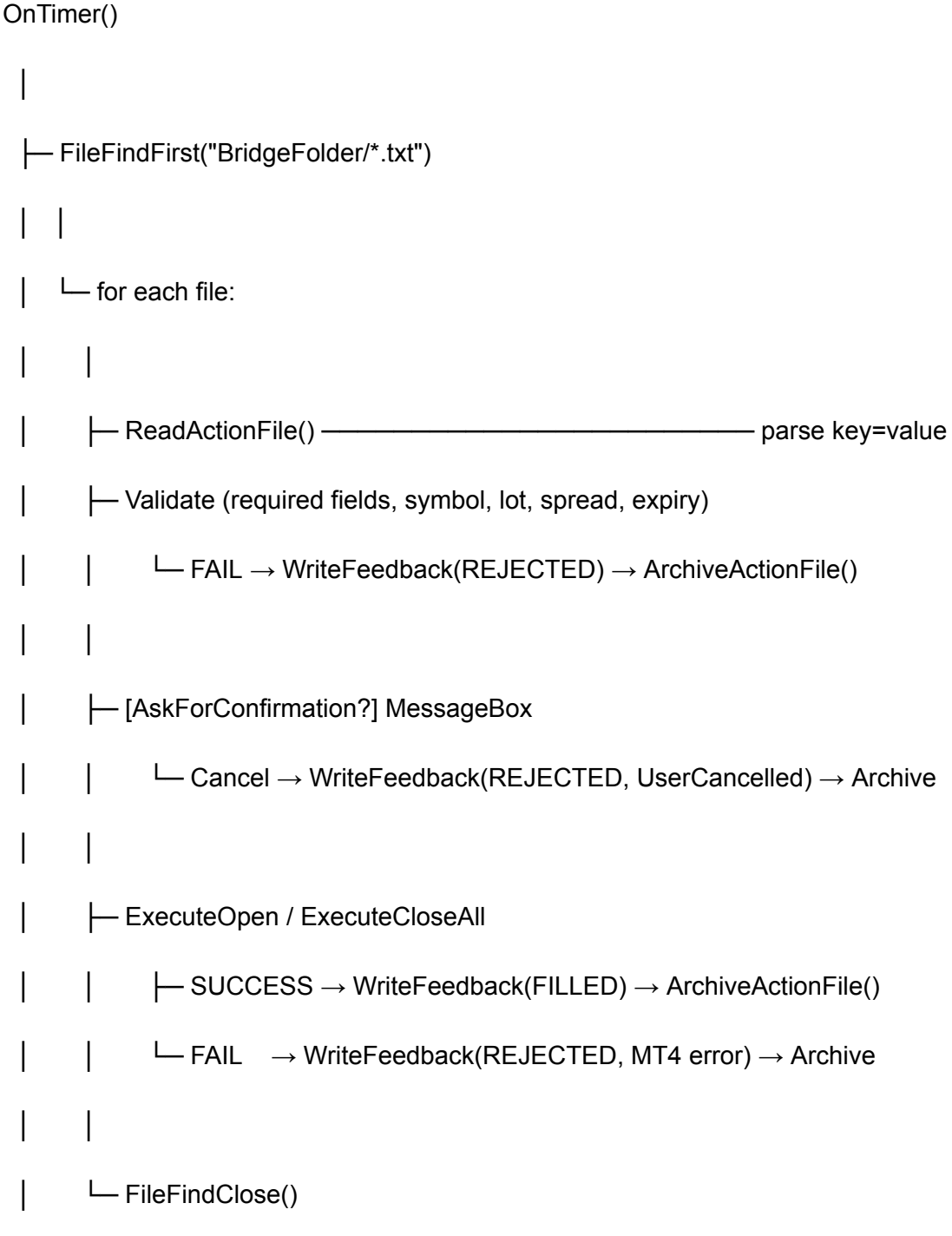
outgoing\ ← Python / manual drops action files here

incoming\ ← EA writes feedback files here

archive\ ← EA moves processed action files here

MT4 file I/O paths are relative to `MQL4\Files\` by default. Use absolute paths (as configured in inputs) with the `FILE_COMMON` flag or place folders inside the MT4 data directory. Recommended: use full Windows paths and ensure the MT4 terminal has write access.

Sequence Diagram



Future Extension Points

Feature	How to Add
`CLOSE_WINNERS` / `CLOSE_LOSERS`	Add `action` branch in `ProcessActionFile`, filter by `OrderProfit()`
Limit/Stop orders	Extend `ExecuteOpen` to handle `OP_BUYLIMIT`, `OP_SELLSTOP`, etc.
Trailing stops	Add `OnTick` handler or separate trailing EA
TradingView webhooks	Python webhook receiver writes same action files; EA unchanged
MT5 port	Rewrite with `trade.Buy()` / `CTrade` — file protocol identical
Partial close	Add `CLOSE_PARTIAL` action with `size` field